Photography Portfolio & CMS

A production-ready, full-stack photography portfolio and content management system built with Next.js 16, React 19, MongoDB, and Cloudinary. Includes a polished public website and a powerful admin dashboard.

📋 Version 1.0.0 📷 Next.js 16 + React 19 🍃 MongoDB + Mongoose ☁️ Cloudinary

Overview

Photography Portfolio & CMS is a complete, full-stack web application built specifically for photographers, studios, and creative professionals. It ships with everything you need out of the box:

- A beautifully designed **public website** with a homepage, photo gallery with category filters, photography stories, about page, and a contact form.
- A feature-rich **Admin Dashboard** for managing gallery photos, categories, stories, team members, testimonials, banners, services, contact submissions, and all site settings — without touching any code.
- A robust **REST API layer** with JWT-based authentication, Zod validation, and a consistent response format.
- **Cloudinary** integration for image hosting and CDN delivery, configured entirely from the admin panel.
- **MongoDB** as the database with Mongoose ODM, optimised for serverless environments with connection pooling.

Features

Public Website

Photography Gallery

Category-filtered photo gallery with real-time filtering tabs and drag-and-drop reordering in the admin.

Photography Stories

Rich story pages with camera metadata (lens, shutter speed, mood), multiple photo galleries, and SEO slugs.

About & Team

Studio about page, sortable team member profiles with designations and photos.

Services Showcase

Visual service cards with images and descriptions, fully managed from the admin dashboard.

Testimonials Carousel

Client testimonials in an Embla-powered carousel with drag-and-drop ordering.

Contact Form

Client inquiry form with category selection, phone input, and message field. All submissions viewable in admin.

SEO Ready

Configurable metadata, Open Graph image, keywords, and page titles managed from the admin settings panel.

Responsive Design

Fully responsive across all screen sizes with Tailwind CSS v4 and shadcn/ui components.

Admin Dashboard

Secure Authentication

JWT-based admin login with bcrypt password hashing and NextAuth v5 session management.

Rich Text Editor

Story content powered by SunEditor — a full-featured WYSIWYG HTML editor.

Drag-and-Drop Ordering

Reorder gallery photos, categories, banners, team, testimonials, and services with @dnd-kit.

Image Upload

Direct Cloudinary uploads from every admin form — no manual image management needed.

Settings Panel

Configure company info, social links, SEO metadata, Cloudinary credentials, and terms & policy from one place.

Dashboard Statistics

At-a-glance stats: total gallery items, stories, contact submissions, team members, and testimonials.

Technology Stack

Next.js 16

App Router, Server Components, Server Actions

React 19

Latest stable release

TypeScript 5

Strict type safety throughout

MongoDB 6+

via Mongoose 9 ODM

NextAuth v5

Session + JWT authentication

Tailwind CSS v4

Utility-first styling

Radix UI + shadcn

Accessible UI primitives

React Hook Form

Performant forms

Zod

Schema-first validation

TanStack Table

Headless data tables

@dnd-kit

Drag-and-drop sorting

SunEditor

Rich text editor

Cloudinary

Media storage & delivery

Zustand

Lightweight state management

Embla Carousel

Smooth carousels

React Photo Album

Masonry photo layouts

Flatpickr

Date & time picker

React Hot Toast

Toast notifications

Lucide Icons

Modern SVG icon set

Moment Timezone

Date & timezone utilities

Requirements

Requirement	Minimum Version	Notes
Node.js	18.17 or later	LTS version recommended
npm	9.x or later	`npm install -g npm`
MongoDB	6.0 or later	Local or MongoDB Atlas (free tier)
Cloudinary Account	Any	Free tier is sufficient

Installation & Setup

Step 1 — Extract Files

```
unzip template-photography.zip -d template-photography
cd template-photography
```

Step 2 — Install Dependencies

```
npm install
```

Step 3 — Configure Environment Variables

```
cp .env.example .env
# Then edit .env with your actual values
```

See the Environment Variables section for all required values.

Step 4 — Set Up MongoDB

Create a free cluster on **MongoDB Atlas**, create a database user with read/write access, whitelist your IP (or `0.0.0.0/0` for all), and copy the connection string into `MONGODB_URI`.

Step 5 — Create Initial Admin User

Connect to your MongoDB database (via Atlas UI or Compass) and insert a document into the `users` collection:

```
{
  "name": "Admin",
  "email": "admin@example.com",
  "password": "<bcrypt-hashed-password>",
  "role": "admin"
}
```

> ℹ️ **Generating a bcrypt hash**
>
> Use an online bcrypt generator or run `node -e "const b=require('bcrypt');b.hash('yourpassword',10).then(console.log)"` in a terminal with bcrypt installed.

Step 6 — Configure Cloudinary (After First Login)

Log into the admin dashboard and navigate to **Admin** → **Settings** → **Cloudinary**. Enter your Cloud Name, API Key, API Secret, and folder name, then save.

Step 7 — Start Development Server

```
npm run dev
```

Open **http://localhost:3000** for the portfolio site. The admin dashboard is at **http://localhost:3000/admin/login**. Documentation is at **http://localhost:3000/documentation**.

Environment Variables

Create a `.env` file in the project root with the following variables:

```
# ── App ──────────────────────────────────────────────────────
# Public base URL used for internal API calls
NEXT_PUBLIC_BASE_URL=http://localhost:3000

# ── Authentication ───────────────────────────────────────────
# Set true when running behind a proxy or on platforms like Vercel
AUTH_TRUST_HOST=true

# NextAuth callback URL — must match your deployment domain
NEXTAUTH_URL=http://localhost:3000

# Secret key for signing JWT tokens
# Generate: openssl rand -base64 32
NEXTAUTH_SECRET=your_strong_random_secret_min_32_chars

# ── Database ─────────────────────────────────────────────────
MONGODB_URI=mongodb+srv://<user>:<password>@cluster.mongodb.net/photography
```

> ☁️ **Cloudinary Credentials**
>
> The `cloud_name`, `api_key`, and `api_secret` are stored and managed through **Admin Dashboard → Settings → Cloudinary** (saved in the database). They do not need to be set as environment variables.

> ⚠️ **Security Notice**
>
> Never commit `.env` to version control. Add it to `.gitignore`.

Running the Application

Command	Description
`npm run dev`	Start development server with Turbopack at `localhost:3000`
`npm run build`	Create an optimised production build
`npm start`	Serve the production build
`npm run lint`	Run ESLint code quality checks
`npm run lint:fix`	Auto-fix ESLint issues
`npm run format`	Format all files with Prettier
`npm run type-check`	TypeScript type checking (no emit)

Project Structure

```
template-photography/
├── app/                            # Next.js App Router root
│   ├── layout.tsx                  # Root layout (fonts, providers)
│   ├── globals.css
│   ├── documentation/              # /documentation route
│   │   └── page.tsx
│   ├── (public)/                   # Public-facing pages (no auth)
│   │   ├── layout.tsx              # Header + Footer wrapper
│   │   ├── page.tsx                # Homepage  "/"
│   │   ├── gallery/page.tsx        # "/gallery"
│   │   ├── about/page.tsx          # "/about"
│   │   ├── stories/page.tsx        # "/stories"
│   │   ├── stories/[slug]/page.tsx # "/stories/:slug"
│   │   └── lets-connect/page.tsx   # "/lets-connect"
│   ├── (admin)/admin/login/        # "/admin/login"
│   └── (private)/admin/dashboard/  # Protected admin (95+ pages)
│       ├── page.tsx                # Dashboard home
│       ├── gallery/
│       ├── stories/
│       ├── banner/
│       ├── categories/
│       ├── teams/
│       ├── testimonial/
│       ├── contact-us/
│       ├── service-gallery/
│       └── settings/
│
├── api/                            # REST API routes
│   ├── public/                     # No authentication required
│   │   ├── gallery/route.ts
│   │   ├── story/route.ts
│   │   ├── story/[slug]/route.ts
│   │   ├── category/route.ts
│   │   ├── banner/route.ts
│   │   ├── team/route.ts
│   │   ├── testimonial/route.ts
│   │   ├── contact-us/route.ts
│   │   ├── service-gallery/route.ts
│   │   ├── home-section/name/[slug]/route.ts
│   │   └── settings/route.ts
│   └── admin/                      # JWT Bearer token required
│       ├── auth/                   # login, me, password
│       ├── gallery/                # CRUD + sort + status
│       ├── banner/
│       ├── category/
│       ├── story/
│       ├── team/
│       ├── testimonial/
│       ├── contact-us/
│       ├── service-gallery/
│       ├── home-section/
```

```
│       ├── file/route.ts          # Cloudinary upload
│       ├── settings/               # general, metadata, cloudinary, terms
│       └── dashboard/stats/route.ts
│
├── components/
│   ├── ui/                         # shadcn/ui base components
│   ├── features/                   # Feature-specific sections
│   │   ├── landing/                # Homepage sections
│   │   ├── about-us/               # About page sections
│   │   ├── our-works/              # Gallery display
│   │   ├── image-gallery/          # Full gallery with filter
│   │   ├── story/                  # Story detail sections
│   │   ├── contact-us-section/     # Contact form & info
│   │   ├── offer-section/          # Services section
│   │   └── frame-felt/             # Testimonials carousel
│   └── form/                       # Reusable form fields
│
├── actions/                        # Next.js Server Actions (by feature)
├── model/                          # Mongoose data models
│   ├── Gallery.ts, Category.ts, Stories.ts
│   ├── Banner.ts, Team.ts, User.ts
│   ├── Testimonial.ts, ContactUs.ts
│   ├── ServiceGallery.ts, HomeSection.ts
│   └── Settings.ts
├── store/                          # Zustand state stores
├── lib/                            # Server utilities & helpers
│   ├── api-client.ts               # Fetch wrapper with JWT injection
│   ├── validation-schema.ts        # All Zod validation schemas
│   ├── authenticate.ts             # JWT token verification
│   └── metadata.ts                 # SEO metadata helpers
├── config/
│   ├── database.ts                 # MongoDB connection with caching
│   └── cloudinary.ts               # Cloudinary client initialisation
├── hooks/                          # Custom React hooks
├── providers/                      # Auth & dashboard providers
├── types/                          # Global TypeScript types
├── proxy.ts                        # NextAuth middleware (route protection)
├── next.config.ts
├── tailwind.config.ts
├── tsconfig.json
└── package.json
```

Pages & Routes

Public Pages

Route	Description
`/`	Homepage — hero banner, featured gallery, services, testimonials carousel
`/gallery`	Full gallery with live category filter tabs
`/about`	About page with studio story, team members, why-choose-us
`/stories`	Photography stories listing with featured cards
`/stories/[slug]`	Story detail with rich content, photo gallery, camera metadata
`/lets-connect`	Client inquiry form with category selection
`/admin/login`	Admin authentication page
`/documentation`	Built-in interactive documentation (this page)

Admin Dashboard Pages (Protected)

All admin routes are under `/admin/dashboard/` and require a valid authentication session.

Route	Description
`/admin/dashboard`	Overview statistics and quick navigation
`/admin/dashboard/gallery`	Upload, sort (drag & drop), and manage photos
`/admin/dashboard/categories`	Manage photo categories with auto slug generation
`/admin/dashboard/stories`	Stories list with status and featured toggles
`/admin/dashboard/stories/create`	Create story with rich text editor and camera specs
`/admin/dashboard/stories/create/[id]`	Edit existing story
`/admin/dashboard/banner`	Page banners with ordering and status
`/admin/dashboard/teams`	Team members with designation and photo
`/admin/dashboard/testimonial`	Client testimonials with ordering
`/admin/dashboard/contact-us`	View all contact form submissions
`/admin/dashboard/service-gallery`	Photography services management
`/admin/dashboard/about-us`	About page content management
`/admin/dashboard/settings`	General info, SEO metadata, Cloudinary config, terms & policy

Authentication

The template uses **NextAuth v5** with a JWT strategy. The middleware in `proxy.ts` protects all routes under `/admin/*` and `/api/admin/*`.

Authentication Flow

1. Admin submits email and password at `/admin/login`.
2. Server looks up the user by email and compares the password using **bcrypt**.
3. On success, a JWT access token is returned and stored in the NextAuth session.
4. `lib/api-client.ts` automatically injects the Bearer token into all admin API requests.
5. All `/api/admin/*` routes verify the token before processing the request.
6. Unauthenticated requests to protected routes are redirected to `/admin/login`.

> ✅ **Password Security**
>
> Passwords are hashed with bcrypt at cost factor 10. Raw passwords are never stored. Password changes require verification of the current password via `POST /api/admin/auth/password`.

API Reference — Public Endpoints

All API responses use a consistent envelope format:

```
// Success
{ "success": true, "statusCode": 200, "message": "...", "data": { ... } }

// Error
{ "success": false, "statusCode": 400, "message": "...", "errors": [ ... ] }
```

Gallery

GET `/api/public/gallery`

Returns gallery items. Supports `category` (slug), `featured` (boolean), `page`, and `limit` query parameters.

Stories

GET `/api/public/story`

Returns featured stories for homepage display.

GET `/api/public/story/[slug]`

Returns a single story by its URL slug.

GET `/api/public/story/more/[slug]`

Returns related stories for the "More Stories" section on a story detail page.

Categories & Other Public Endpoints

GET `/api/public/category`

Returns all active categories ordered by position.

`GET` **/api/public/banner**

Returns all active page banners ordered by position.

`GET` **/api/public/team**

Returns all active team members.

`GET` **/api/public/testimonial**

Returns all active testimonials ordered by position.

`GET` **/api/public/service-gallery**

Returns all active service gallery items.

`POST` **/api/public/contact-us**

Submit a contact form inquiry.

```
{ "name": "Alice Johnson", "email": "alice@example.com",
  "phone": "+1234567890", "message": "I'm interested in wedding photography.",
  "category": "Wedding" }
```

`GET` **/api/public/settings**

Returns public site settings (company info, social links, SEO metadata).

`GET` **/api/public/home-section/name/[slug]**

Returns a specific home page section by its slug key (e.g. `hero`, `about`, `services`).

API Reference — Admin Authentication

All admin management endpoints require: `Authorization: Bearer <token>`

POST `/api/admin/auth/login`

Authenticates an admin user and returns a JWT token.

```
// Request
{ "email": "admin@example.com", "password": "your_password" }

// Response
{
  "success": true,
  "data": {
    "token": "eyJhbGci...",
    "user": { "name": "Admin", "email": "admin@example.com", "role": "admin" }
  }
}
```

GET `/api/admin/auth/me`

Returns the currently authenticated admin's profile. Requires Bearer token.

POST `/api/admin/auth/password`

Changes the admin password. Requires Bearer token.

```
{ "currentPassword": "old_password", "newPassword": "new_password" }
```

API Reference — Gallery

Method	Endpoint	Description
POST	`/api/admin/gallery`	Create a new gallery item
GET	`/api/admin/gallery`	List all gallery items (paginated)
PUT	`/api/admin/gallery/[id]`	Update gallery item by ID
DELETE	`/api/admin/gallery/[id]`	Delete gallery item by ID
PATCH	`/api/admin/gallery/status/[id]`	Toggle active status / featured flag
POST	`/api/admin/gallery/sort`	Reorder gallery items

```
// Create Gallery Item — Request Body
{
  "image": "https://res.cloudinary.com/...",
  "title": "Golden Hour Portrait",
  "author": "Jane Smith",
  "category": "portrait",
  "status": true,
  "isFeatured": false
}

// Sort — Request Body
{ "items": [{ "id": "item_id", "orderBy": 1 }, ...] }
```

API Reference — Categories

Method	Endpoint	Description
POST	`/api/admin/category`	Create category
GET	`/api/admin/category`	List all categories
PUT	`/api/admin/category/[id]`	Update category
DELETE	`/api/admin/category/[id]`	Delete category
PATCH	`/api/admin/category/status/[id]`	Toggle status
POST	`/api/admin/category/sort`	Reorder categories

```
// Create Category — Request Body
{
  "name": "Portrait",
  "slug": "portrait",
  "status": true,
  "featured": false,
  "parent": null  // ObjectId for sub-category, null for root
}
```

API Reference — Stories

Method	Endpoint	Description
POST	`/api/admin/story`	Create a story
GET	`/api/admin/story`	List all stories (paginated)
PUT	`/api/admin/story/[id]`	Update story
DELETE	`/api/admin/story/[id]`	Delete story
PATCH	`/api/admin/story/status/[id]`	Toggle status / featured

```
// Create Story — Request Body
{
  "title": "Golden Sunset Wedding",
  "slug": "golden-sunset-wedding",
  "description": "<p>Rich HTML content from SunEditor...</p>",
  "image": "https://res.cloudinary.com/...",
  "photos": ["url1", "url2"],
  "client": "Sarah & James",
  "photographer": "Jane Smith",
  "mood": "Romantic",
  "category": ["categoryObjectId"],
  "camera": "Sony A7R IV",
  "shutterSpeed": "1/500s",
  "lensUsed": "85mm f/1.4",
  "focus": ["Natural light", "Bokeh"],
  "approach": ["Candid moments", "Golden hour"],
  "eventDate": "2024-06-15",
  "isFeatured": true,
  "status": true
}
```

API Reference — Banner

Method	Endpoint	Description
POST	`/api/admin/banner`	Create a new banner
GET	`/api/admin/banner`	List all banners
PUT	`/api/admin/banner/[id]`	Update banner by ID
DELETE	`/api/admin/banner/[id]`	Delete banner by ID
PATCH	`/api/admin/banner/status/[id]`	Toggle active status
POST	`/api/admin/banner/sort`	Reorder banners

```
// Create Banner — Request Body
{
  "sectionName": "gallery-hero",
  "heading": "Our Photography",
  "shortDesc": "Capturing moments that last a lifetime",
  "categories": "Wedding, Portrait",
  "position": 1,
  "status": true
}
```

API Reference — Team

Method	Endpoint	Description
POST	`/api/admin/team`	Create team member
GET	`/api/admin/team`	List all team members
PUT	`/api/admin/team/[id]`	Update team member
DELETE	`/api/admin/team/[id]`	Delete team member
PATCH	`/api/admin/team/status/[id]`	Toggle status
POST	`/api/admin/team/sort`	Reorder team

```
// Create Team Member — Request Body
{
  "name": "Jane Smith",
  "designation": "Lead Photographer",
  "image": "https://res.cloudinary.com/...",
  "status": true
}
```

API Reference — Testimonial

Method	Endpoint	Description
POST	`/api/admin/testimonial`	Create testimonial
GET	`/api/admin/testimonial`	List all testimonials
PUT	`/api/admin/testimonial/[id]`	Update testimonial
DELETE	`/api/admin/testimonial/[id]`	Delete testimonial
PATCH	`/api/admin/testimonial/status/[id]`	Toggle status
POST	`/api/admin/testimonial/sort`	Reorder testimonials

```
// Create Testimonial — Request Body
{
  "quote": "The photos were absolutely stunning. Highly recommended!",
  "authorName": "Sarah Johnson",
  "authorRole": "Wedding Client",
  "order": 1,
  "status": true
}
```

API Reference — Contact

Method	Endpoint	Description
GET	`/api/admin/contact-us`	List all contact submissions
DELETE	`/api/admin/contact-us`	Delete submission(s)

API Reference — Service Gallery

Method	Endpoint	Description
POST	`/api/admin/service-gallery`	Create service
GET	`/api/admin/service-gallery`	List all services
PUT	`/api/admin/service-gallery/[id]`	Update service
DELETE	`/api/admin/service-gallery/[id]`	Delete service
PATCH	`/api/admin/service-gallery/status/[id]`	Toggle status
POST	`/api/admin/service-gallery/sort`	Reorder services

```
// Create Service — Request Body
{
  "heading": "Wedding Photography",
  "shortDesc": "Timeless wedding moments captured with elegance.",
  "image": "https://res.cloudinary.com/...",
  "position": 1,
  "status": true
}
```

API Reference — Home Sections

Method	Endpoint	Description
`GET`	`/api/admin/home-section`	List all home sections
`POST`	`/api/admin/home-section`	Create home section
`GET`	`/api/admin/home-section/name/[slug]`	Get section by key

Section Keys: `hero`, `about`, `services`, `gallery`, `testimonials`, `cta`

API Reference — File Upload

POST `/api/admin/file`

Upload a file to Cloudinary. Request must use `multipart/form-data`. Requires Bearer token.

Field	Type	Required	Description
`file`	File	Yes	Image file (JPEG, PNG, WebP, AVIF — max 20 MB)
`folder`	string	No	Cloudinary subfolder name

```
// Response
{
  "success": true,
  "url": "https://res.cloudinary.com/your-cloud/image/upload/v1234/folder/file.jpg",
  "publicId": "folder/file"
}
```

API Reference — Settings

Method	Endpoint	Description
GET	`/api/admin/settings`	Fetch all settings
POST	`/api/admin/settings/general`	Update general settings (company info, social links)
POST	`/api/admin/settings/metadata`	Update SEO metadata (title, description, keywords, OG image)
POST	`/api/admin/settings/cloudinary`	Update Cloudinary configuration
POST	`/api/admin/settings/page-banner`	Update per-page banner settings
POST	`/api/admin/settings/terms`	Update terms & privacy policy (HTML content)

API Reference — Dashboard Stats

GET `/api/admin/dashboard/stats`

Returns aggregate counts for the dashboard overview. Requires Bearer token.

```
// Response
{
  "success": true,
  "data": {
    "gallery": 124,
    "stories": 18,
    "contacts": 45,
    "teams": 6,
    "testimonials": 12
  }
}
```

Database Models

User

```
{ name: String, email: String (unique), password: String (bcrypt), role: "admin" |
"user" }
```

Gallery

```
{ image: String, title: String, author: String, category: String,
  status: Boolean, isFeatured: Boolean, orderBy: Number }
```

Category

```
{ name: String, slug: String (unique), status: Boolean, featured: Boolean,
  position: Number, parent: ObjectId (nullable) }
```

Stories

```
{ title: String, slug: String (unique), description: String (HTML),
  image: String, photos: [String], client: String, photographer: String,
  mood: String, category: [ObjectId], camera: String, shutterSpeed: String,
  lensUsed: String, focus: [String], approach: [String],
  eventDate: Date, isFeatured: Boolean, status: Boolean }
```

Team

```
{ name: String, designation: String, image: String, status: Boolean }
```

Testimonial

```
{ quote: String, authorName: String, authorRole: String, order: Number, status:
Boolean }
```

ContactUs

```
{ name: String, email: String, phone: String, message: String, category: String,
status: Boolean }
```

ServiceGallery

```
{ heading: String, shortDesc: String, image: String, position: Number, status:
Boolean }
```

Settings (single document)

```
{ general: { companyName, companyPhone, companyAddress, supportEmail, ownerName,
ownerEmail,
              logo, favicon, facebook, instagram, twitter, youtube },
  cloudinary: { cloudName, apiKey, apiSecret, folder, secureUrlBase },
  metadata:   { title, applicationName, description, keywords: [String],
openGraphImage },
  termsPolicy:{ terms: String (HTML), policy: String (HTML) },
  businessHours: [{ dayOfWeek: 0-6, openTime: Number, closeTime: Number, isClosed:
Boolean }] }
```

Image Management

Upload Flow

1. Admin selects file(s) in any dashboard form.
2. Frontend sends the file to `POST /api/admin/file`.
3. Server streams the file to Cloudinary via `upload_stream`.
4. Cloudinary returns a secure CDN URL.
5. The URL is stored in the MongoDB document field.

Cloudinary Configuration

Credentials are loaded **dynamically** from the `Settings` database document — not from environment variables. This enables live configuration changes from the admin dashboard without redeployment.

To configure: Log in → **Admin** → **Settings** → **Cloudinary** → enter Cloud Name, API Key, API Secret, and folder name → Save.

> ✅ **Supported Formats**
>
> JPEG, PNG, WebP, AVIF, GIF. Maximum file size is
>
> **20 MB**
>
> , set via `serverActions.bodySizeLimit` in `next.config.ts`.

Deployment Guide

Vercel (Recommended)

```
# 1. Push your project to a GitHub repository
git push origin main

# 2. Import at vercel.com → New Project → select your repo
# 3. Add environment variables in Vercel project settings
# 4. Deploy — Vercel rebuilds automatically on every push
```

Self-Hosted (Node.js)

```
# Build the application
npm run build

# Start production server (runs on port 3000)
npm start
```

Nginx Reverse Proxy

```
server {
    listen 80;
    server_name yourdomain.com;

    location / {
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_cache_bypass $http_upgrade;
    }
}
```

> ✅ **Production Checklist**
>
> Update `NEXTAUTH_URL` and `NEXT_PUBLIC_BASE_URL` to your production domain. Set `AUTH_TRUST_HOST=true` when running behind a proxy or on Vercel. Use a strong, random `NEXTAUTH_SECRET` (minimum 32 characters).

Changelog

v1.0.0 — Initial Release

- Full photography portfolio with public website and admin CMS
- MongoDB + Mongoose database layer with serverless-optimised connection pooling
- Cloudinary dynamic image hosting (credentials managed from admin panel)
- NextAuth v5 JWT authentication with bcrypt password hashing
- Gallery with category filtering and drag-and-drop reordering
- Stories / blog module with SunEditor rich text, multi-photo upload, and camera metadata
- Team, testimonial, banner, and service gallery management
- Site settings: general info, SEO metadata, Cloudinary, page banners, terms & policy
- Contact form with admin review and delete interface
- Built-in documentation page at `/documentation`
- Full TypeScript coverage with Zod schema validation on all API routes
- Tag-based Next.js cache invalidation for instant content updates